

Universidad San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias y Sistemas
Intr. a la Programación y Computación 2
Ing. Daniel Gonzales



MANUAL TÉCNICO PROYECTO FINAL

Henry Josué Argueta Champet
Reg. academico: 202131261
Fecha: 09/05/2025

ÍNDICE

Descripción.....	3
Objetivos.....	3
Descripción del Sistema.....	4
Tipos de Usuario.....	4
Estados del Proyecto.....	4
Herramientas y Tecnologías.....	5
Backend.....	5
Frontend.....	5
Arquitectura del Sistema.....	6
Arquitectura General.....	6
Diagrama de Paquetes — Backend.....	7
Diagrama de Paquetes — Frontend.....	8
Diseño de Base de Datos.....	9
Diagrama Entidad/Relación — Tablas Principales.....	9
Relaciones Principales.....	10
Diagrama Tablas.....	10
Documentación de la API REST.....	12
Convenciones Generales.....	12

Descripción.

Este documento ha sido creado con un propósito claro: servir como una guía detallada y completa para todos los colaboradores que deseen comprender, analizar y profundizar en el código fuente de nuestro programa. Mi objetivo es que, a través de esta explicación, puedan navegar por la estructura del proyecto con total confianza, entendiendo no sólo qué hace cada componente, sino también el porqué detrás de las decisiones de diseño y la lógica implementada.

Aquí, desglosaremos cada sección del código, explicando su funcionalidad, las interacciones entre los módulos y los patrones de diseño utilizados. Esto no solo facilitará la incorporación de nuevos miembros al equipo, sino que también permitirá una colaboración más eficiente en futuras mejoras y actualizaciones.

Objetivos.

Este manual técnico ha sido diseñado como una guía esencial para ayudar a comprender y dominar todos los aspectos del software. El objetivo principal es que, como usuario, sientas completamente la capacidad para utilizar, mantener y configurar el sistema de manera eficiente.

- **Comprender el sistema:** Obtener una visión completa de la definición, el diseño, la estructura y la organización del software.
- **Gestionar el sistema:** Acceder a instrucciones claras y detalladas para configurar, mantener y resolver cualquier problema que pueda surgir.
- **Ser un usuario experto:** Familiar al usuario con las herramientas y funciones clave que te permitirán administrar, editar y configurar el software de manera efectiva.

Descripción del Sistema

ConnectWork es una plataforma web que conecta clientes que necesitan servicios digitales con freelancers que los ofrecen. Actúa como intermediario gestionando el ciclo completo de los proyectos: desde su publicación hasta la entrega final, incluyendo propuestas, contratos y pagos.

El sistema fue desarrollado como Proyecto Final de IPC2 (Primer Semestre 2026), implementando una arquitectura de dos capas completamente separadas: backend REST con Java/Jakarta EE Servlets en Apache Tomcat, y frontend SPA con Angular 21 y TypeScript estricto.

Tipos de Usuario

Rol	Descripción
ADMIN	Gestiona categorías, habilidades, usuarios y comisiones. No se registra públicamente; existe un admin precargado.
CLIENTE	Publica proyectos, gestiona propuestas, revisa entregas, califica freelancers y recarga saldo.
FREELANCER	Explora proyectos abiertos, envía propuestas, sube entregas y solicita nuevas habilidades al catálogo.

Estados del Proyecto

El ciclo de vida de un proyecto sigue estos estados

Estado	Descripción
ABIERTO	Proyecto publicado, recibiendo propuestas de freelancers.
EN_REVISION	Cliente seleccionó una propuesta, confirmando el contrato.
EN_PROGRESO	Contrato activo; el freelancer está trabajando en el proyecto.
ENTREGA_PENDIENTE	Freelancer subió entrega; el cliente debe aprobar o rechazar.
COMPLETADO	Cliente aprobó la entrega final; pago liberado al freelancer.
CANCELADO	Cliente canceló el contrato; monto bloqueado devuelto a su saldo.

Herramientas y Tecnologías

Backend

Tecnología	Versión	Propósito
Java	21 (LTS)	Lenguaje de programación del backend.
Jakarta EE Servlets	5.0.0	Tecnología base para los endpoints REST.
Apache Tomcat	10.x	Servidor de despliegue del archivo WAR.
Maven	3.x	Gestión de dependencias y build del proyecto.
MySQL / MariaDB	8.0	Sistema gestor de base de datos relacional.
MySQL Connector/J	8.0.33	Driver JDBC para conexión con MySQL.
Gson	2.10.1	Serialización/deserialización Java ↔ JSON.
jBCrypt	0.4	Encriptación segura de contraseñas (hash).
JWT	0.11.5	Generación y validación de JSON Web Tokens.
iTextPDF	5.5.13.3	Exportación de reportes a formato PDF.
SweetAlert2	npm	Alertas y modales visuales en el frontend.

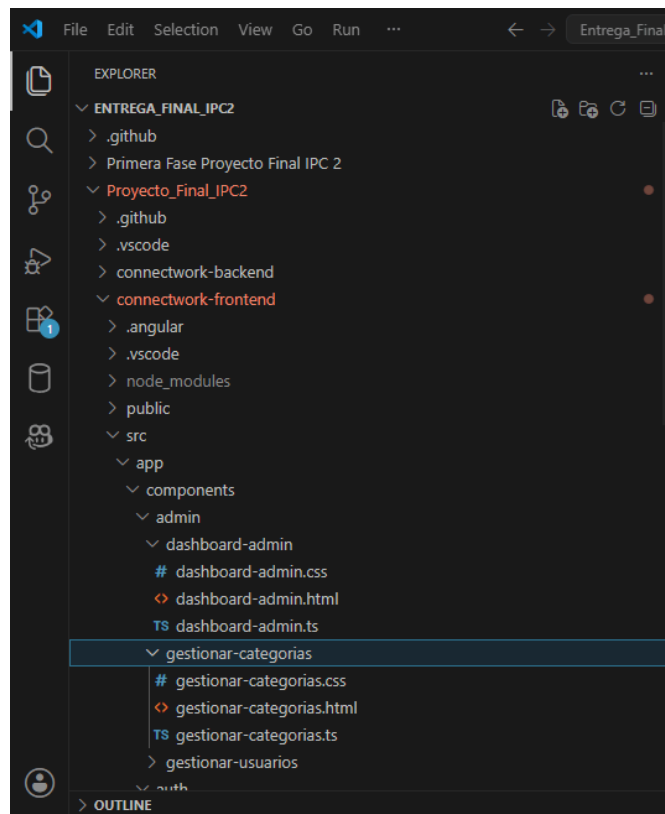
Frontend

Tecnología	Versión	Propósito
Angular	21.2.10	Framework principal SPA del frontend.
TypeScript	Estricto	Lenguaje tipado para el desarrollo Angular.
HttpClient	Angular	Consumo de la API REST (HTTP/JSON).
Angular Router	Angular	Navegación SPA y protección de rutas.
Angular Guards	Angular	Control de acceso por rol (authGuard, adminGuard, clienteGuard, freelancerGuard).
Angular Services	Angular	Inyección de dependencias y lógica HTTP compartida.
SweetAlert2	npm	Alertas, confirmaciones y modales amigables.

Arquitectura del Sistema

Arquitectura General

ConnectWork implementa una arquitectura cliente-servidor con separación estricta entre frontend y backend. Los componentes se comunican exclusivamente a través de HTTP/REST con respuestas en formato JSON.

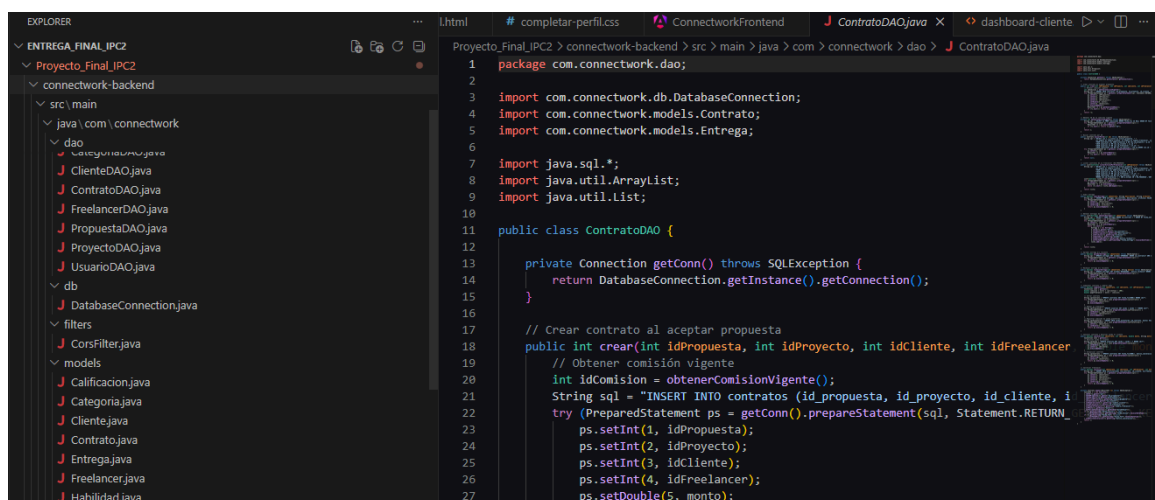


Capa	Descripción
Frontend SPA (Angular)	Corre en el navegador del usuario. Puerto 4200 en desarrollo. Consume la API REST del backend mediante HttpClient con token JWT en el header Authorization.
Backend API REST (Java)	Desplegado como WAR en Tomcat. Puerto 8080. Expone todos los endpoints bajo /connectwork-backend/api/**. Responde JSON.
Base de datos (MySQL)	Corre localmente en puerto 3306. Base de datos: connectwork_2. Accedida únicamente desde el backend mediante JDBC.

Diagrama de Paquetes — Backend

El proyecto Maven sigue la estructura de paquetes estándar com.connectwork:

Paquete	Contenido y responsabilidad
com.connectwork.models	Clases POJO (Entidades): Usuario, Cliente, Freelancer, Proyecto, Propuesta, Contrato, Entrega, Calificacion, Categoria, Habilidad.
com.connectwork.dao	Capa de acceso a datos (patrón DAO): UsuarioDAO, ClienteDAO, FreelancerDAO, ProyectoDAO, PropuestaDAO, ContratoDAO, CategoriaDAO.
com.connectwork.servlets	Endpoints REST (Jakarta Servlets): AuthServicelet, ProyectoServlet, PropuestaServlet, ContratoServlet, ClienteServlet, FreelancerServlet, CategoriaServlet, UsuarioServlet.
com.connectwork.filters	CorsFilter: intercepta todas las peticiones (/*) y agrega headers CORS para permitir comunicación desde Angular en puerto 4200.
com.connectwork.utils	JwtUtil: generación, validación y extracción de datos de JSON Web Tokens. Expiración: 24 horas. Algoritmo: HMAC-SHA256.
com.connectwork.db	DatabaseConnection: patrón Singleton para la conexión JDBC con MySQL. Se reconecta automáticamente si la conexión está cerrada.

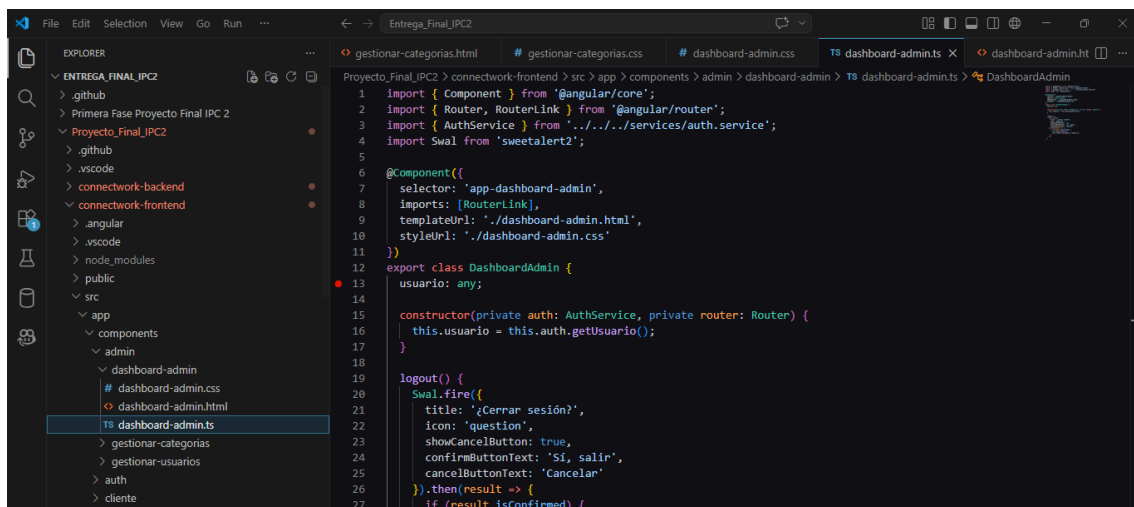


The screenshot shows an IDE with two main panes. The left pane, labeled 'EXPLORER', displays the project structure for 'Proyecto_Final_IPC2'. It shows a 'connectwork-backend' directory containing 'src/main/java/com/connectwork' with subdirectories 'dao', 'db', 'filters', and 'models'. The 'dao' directory lists several DAO classes: 'ContratoDAO.java', 'ClienteDAO.java', 'FreelancerDAO.java', 'PropuestaDAO.java', 'ProyectoDAO.java', and 'UsuarioDAO.java'. The 'db' directory contains 'DatabaseConnection.java'. The 'filters' directory contains 'CorsFilter.java'. The 'models' directory contains 'Calificacion.java', 'Categoria.java', 'Cliente.java', 'Contrato.java', 'Entrega.java', 'Freelancer.java', and 'Habilidad.java'. The right pane shows the code for 'ContratoDAO.java'. It starts with package and import statements, followed by a class definition with a private method 'getConn()' and a public static method 'crear()'.

```
1 package com.connectwork.dao;
2
3 import com.connectwork.db.DatabaseConnection;
4 import com.connectwork.models.Contrato;
5 import com.connectwork.models.Entrega;
6
7 import java.sql.*;
8 import java.util.ArrayList;
9 import java.util.List;
10
11 public class ContratoDAO {
12
13     private Connection getConn() throws SQLException {
14         return DatabaseConnection.getInstance().getConnection();
15     }
16
17     // Crear contrato al aceptar propuesta
18     // Obtener comisión vigente
19     // Obtener comisión vigente
20     int idComision = obtenerComisionVigente();
21     String sql = "INSERT INTO contratos (id_propuesta, id_proyecto, id_cliente, i";
22     try (PreparedStatement ps = getConn().prepareStatement(sql, Statement.RETURN
23         ps.setInt(1, idPropuesta);
24         ps.setInt(2, idProyecto);
25         ps.setInt(3, idCliente);
26         ps.setInt(4, idFreelancer);
27         ps.setDouble(5, monto);
```

Diagrama de Paquetes — Frontend

Carpeta	Contenido
src/app/models/	Interfaces TypeScript: Usuario, Cliente, Freelancer, Proyecto, Propuesta, Contrato, Entrega, Calificacion, Categoria, Habilidad. (models.ts)
src/app/services/	Servicios Angular con inyección de dependencias: AuthService, ProyectoService, PropuestaService, ContratoService, ClienteService, FreelancerService, CategoriaService.
src/app/guards/	Guards de rutas: authGuard (sesión activa), adminGuard (rol ADMIN), clienteGuard (rol CLIENTE), freelancerGuard (rol FREELANCER).
src/app/components/auth/	Componentes públicos: Login, Registro, CompletarPerfil.
src/app/components/cliente/	Módulo cliente: DashboardCliente, MisProyectos, CrearProyecto, VerPropuestas.
src/app/components/freelancer/	Módulo freelancer: DashboardFreelancer, ExplorarProyectos, MisPropuestas, MisContratos.
src/app/components/admin/	Módulo admin: DashboardAdmin, GestionarCategorias, GestionarUsuarios.



```
1 import { Component } from '@angular/core';
2 import { Router, RouterLink } from '@angular/router';
3 import { AuthService } from '.../services/auth.service';
4 import Swal from 'sweetalert2';
5
6 @Component({
7   selector: 'app-dashboard-admin',
8   imports: [RouterLink],
9   templateUrl: './dashboard-admin.html',
10   styleUrls: ['./dashboard-admin.css']
11 })
12
13 export class DashboardAdmin {
14   usuario: any;
15
16   constructor(private auth: AuthService, private router: Router) {
17     this.usuario = this.auth.getUsuario();
18   }
19
20   login() {
21     Swal.fire({
22       title: '¿Cerrar sesión?',
23       icon: 'question',
24       showCancelButton: true,
25       confirmButtonText: 'Sí, salir',
26       cancelButtonText: 'Cancelar'
27     }).then(result => {
28       if (result.isConfirmed) {
```


Diseño de Base de Datos

Diagrama Entidad/Relación — Tablas Principales

Tabla	Descripción y campos principales
usuarios	Tabla central. Campos: id (PK), nombre, username (UNIQUE), password_hash (BCrypt), email (UNIQUE), telefono, direccion, cui, fecha_nac, rol (ENUM: CLIENTE/FREELANCER/ADMIN), activo (BOOL), perfil_completo (BOOL), saldo (DECIMAL), fecha_registro (TIMESTAMP).
clientes	Extiende usuarios. PK compartida con usuarios. Campos adicionales: descripcion, sector, sitio_web (nullable).
freelancers	Extiende usuarios. PK compartida. Campos: bio, experiencia (ENUM: JUNIOR/SEMI_SENIOR/SENIOR), tarifa_hora (DECIMAL).
categorias	Categorías de servicios. Campos: id, nombre, activa (BOOL, default TRUE).
habilidades	Habilidades del catálogo. Campos: id, nombre, descripcion, id_categoria (FK→categorias), activa (BOOL).
freelancer_habilidades	Tabla pivote N:M. Campos: id_freelancer (FK→freelancers), id_habilidad (FK→habilidades). PK compuesta.
proyectos	Proyectos publicados. Campos: id, id_cliente (FK→clientes), titulo, descripcion, id_categoria (FK→categorias), presupuesto_max (DECIMAL), fecha_limite (DATE), estado (ENUM: ABIERTO/EN_REVISION/EN_PROGRESO/ENTREGA_PENDIENTE/COMPLETADO/CANCELADO), fecha_creacion (TIMESTAMP).
proyecto_habilidades	Tabla pivote N:M. Campos: id_proyecto (FK→proyectos), id_habilidad (FK→habilidades). PK compuesta.
propuestas	Propuestas a proyectos. Campos: id, id_proyecto (FK→proyectos), id_freelancer (FK→freelancers), monto_ofertado (DECIMAL), plazo_dias (INT), carta (TEXT), estado (ENUM: PENDIENTE/ACEPTADA/RECHAZADA/RETIRADA), fecha_envio (TIMESTAMP).
comisiones	Historial de porcentajes. Campos: id, porcentaje (DECIMAL), fecha_inicio (TIMESTAMP), fecha_fin (TIMESTAMP, NULL si vigente).
contratos	Contratos generados al aceptar propuesta. Campos: id, id_propuesta (FK), id_proyecto (FK), id_cliente (FK), id_freelancer (FK), monto (DECIMAL), id_comision (FK→comisiones), fecha_inicio (TIMESTAMP), fecha_fin (TIMESTAMP nullable), motivo_cancelacion (TEXT nullable).
entregas	Entregas dentro de un contrato. Campos: id, id_contrato (FK→contratos), descripcion (TEXT), archivos (TEXT, URL/referencia), estado (ENUM: PENDIENTE/APROBADA/RECHAZADA), motivo_rechazo (TEXT nullable), fecha_entrega (TIMESTAMP).
calificaciones	Calificaciones de freelancers. Campos: id, id_contrato (FK), id_cliente (FK), id_freelancer (FK), estrellas (INT 1-5), comentario (TEXT), fecha (TIMESTAMP).
recargas	Historial de recargas de clientes. Campos: id, id_cliente (FK), monto (DECIMAL), fecha (TIMESTAMP).
saldo_plataforma	Comisiones acumuladas. Campos: id, id_contrato (FK), monto (DECIMAL), fecha (TIMESTAMP).

Relaciones Principales

- usuarios → clientes / freelancers: relación 1:1 mediante PK compartida (herencia de tabla por tipo).
- freelancers ↔ habilidades: relación N:M mediante tabla freelancer_habilidades.
- proyectos ↔ habilidades: relación N:M mediante tabla proyecto_habilidades.
- proyectos → propuestas: 1:N (un proyecto recibe múltiples propuestas).
- propuestas → contratos: 1:1 (solo la propuesta aceptada genera contrato).
- contratos → entregas: 1:N (ciclos de entrega/rechazo dentro del mismo contrato).
- contratos → calificaciones: 1:1 (una calificación por contrato completado).
- contratos → comisiones: N:1 (cada contrato toma la comisión vigente al crearse).
- contratos → saldo_plataforma: 1:1 (una entrada de comisión por contrato completado).

Diagrama Tablas

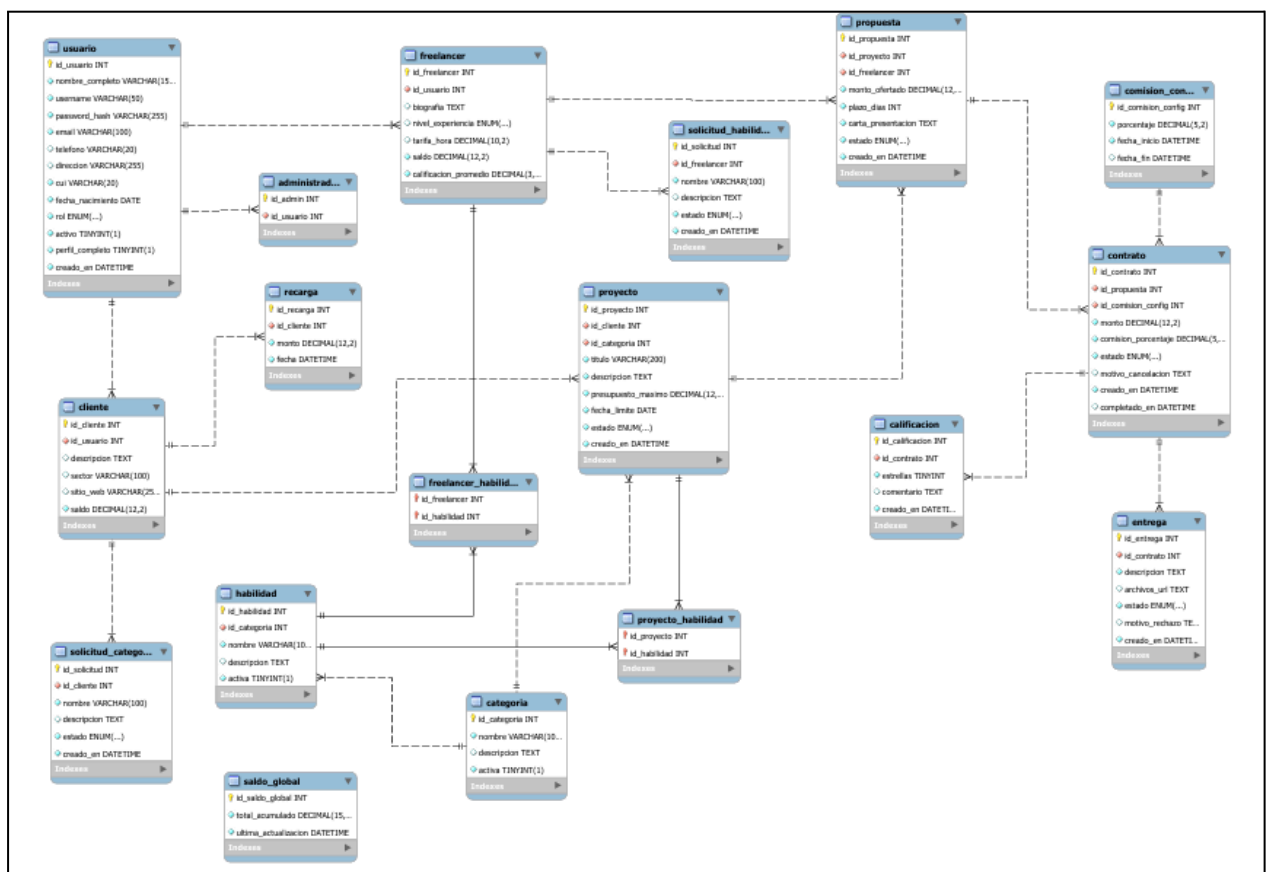
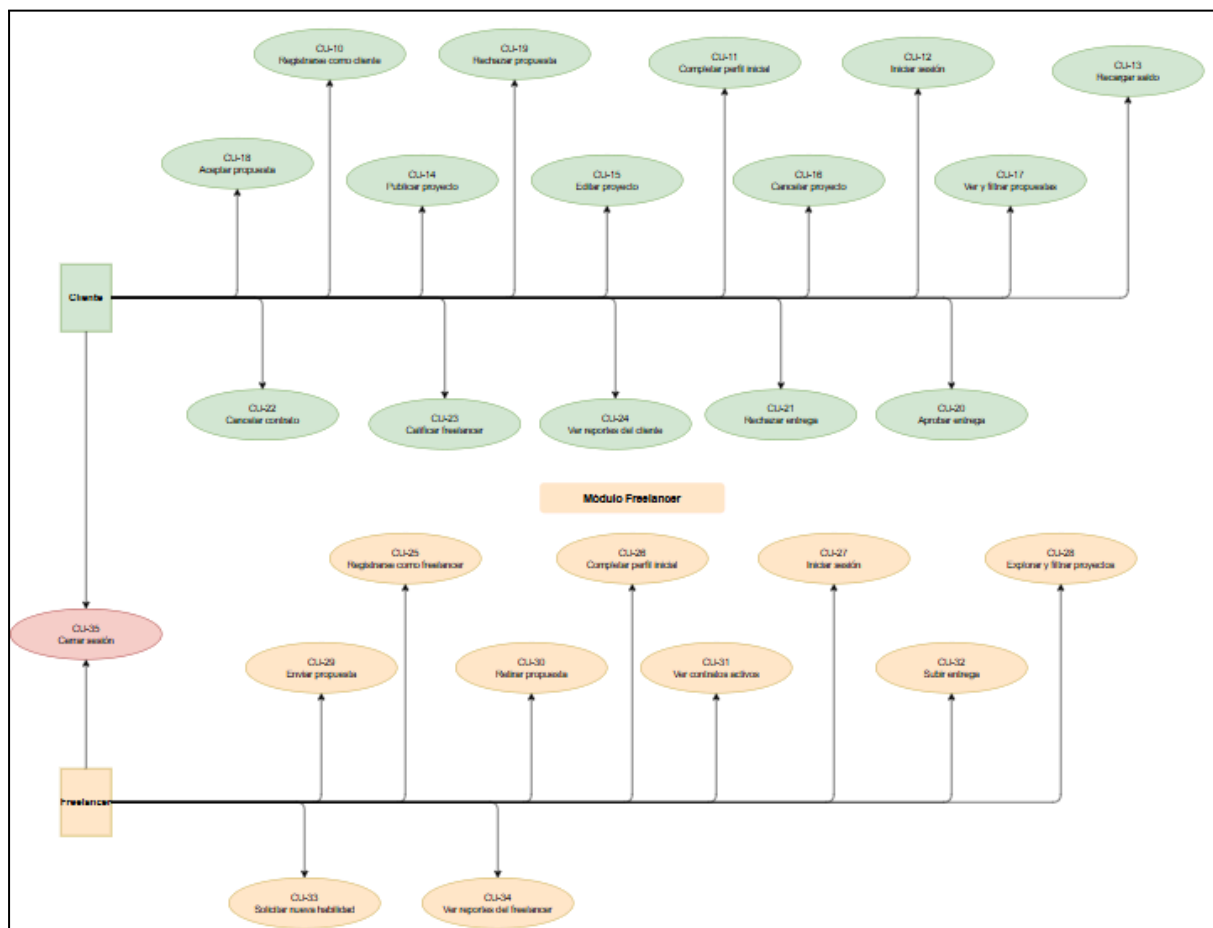
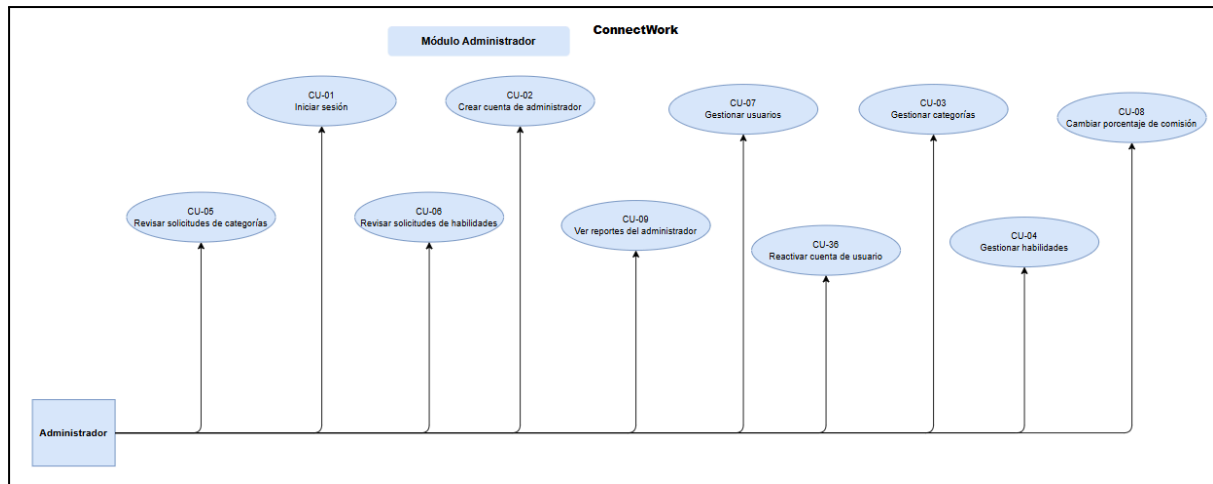


Diagrama de Casos de Uso



Documentación de la API REST

Convenciones Generales

- URL base: `http://localhost:8080/connectwork-backend/api`
- Todas las respuestas son en formato JSON (Content-Type: `application/json`).
- Endpoints protegidos requieren el header: `Authorization: Bearer <token>`
- El token JWT expira en 24 horas. Algoritmo: HMAC-SHA256.
- Errores devuelven: `{ "error": "mensaje descriptivo" }`

```
PS C:\Users\Pablo\OneDrive\Escritorio\Entrega_Final_IPC2> cd "C:\Users\Pablo\OneDrive\Escritorio\Entrega_Final_IPC2\Proyecto_Final_IPC2\connectwork-frontend"
>> ng serve
Initial chunk files | Names      | Raw size
main.js            | main      | 146.20 kB |
styles.css         | styles    | 95 bytes  |

| Initial total | 146.30 kB

Application bundle generation complete. [4.741 seconds] - 2026-05-11T19:18:34.145Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → press h + enter to show help
Initial chunk files | Names | Raw size
main.js            | main  | 152.35 kB |
```